

Design Pattern: “the Law of Demeter”

บรรยายโดย ผศ.ดร.ธราวิเชษฐ์ ธิติจรูญโรจน์

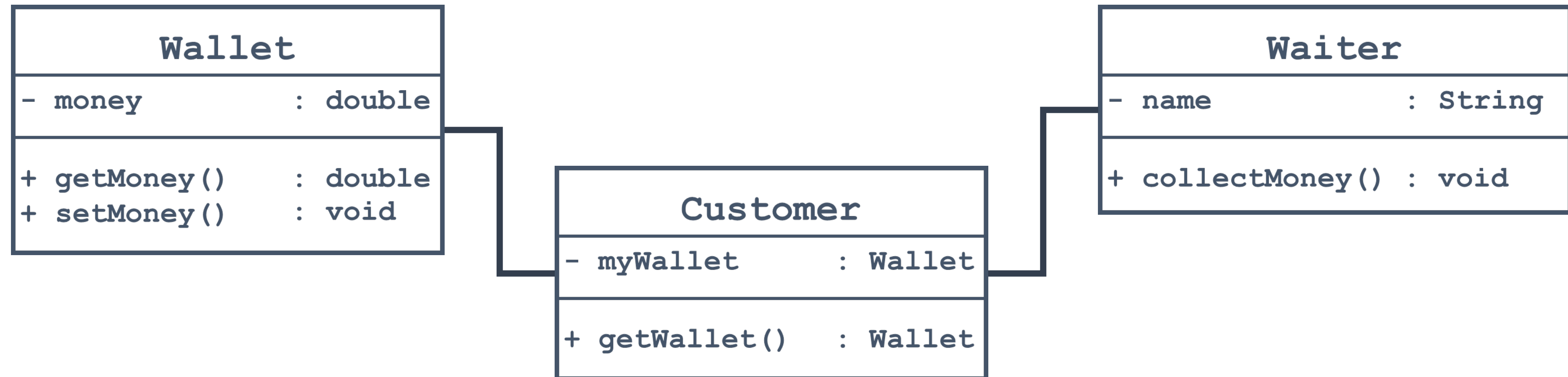
คณะเทคโนโลยีสารสนเทศ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

Design Pattern: “*the Law of Demeter*”



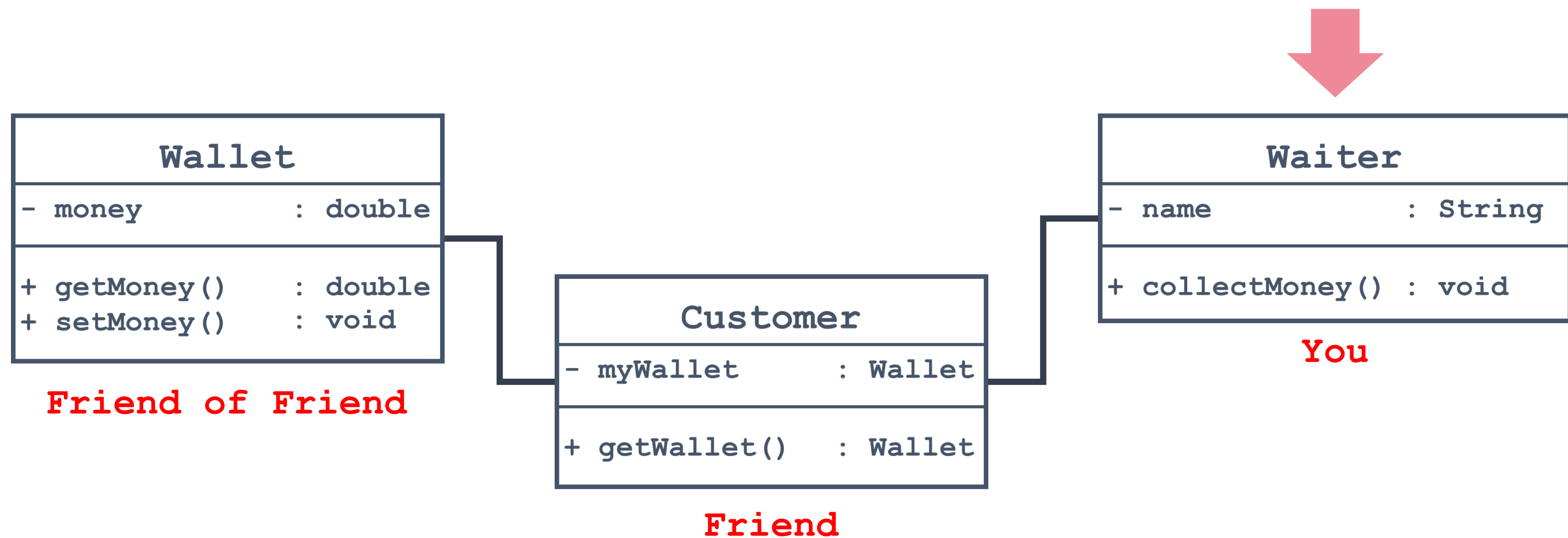
the Law of Demeter คือ Design Pattern รูปแบบหนึ่งซึ่งช่วยให้การออกแบบเชิงวัตถุไม่ผูกติดซึ่งกันและกันมากเกินไป อีกทั้งยังสอดคล้องกับ Information Hiding ที่จะซ่อนข้อมูลจากวัตถุอื่น ๆ ที่ไม่ควรทราบหรือเกี่ยวข้องกัน (แค่เพื่อนเท่านั้น)



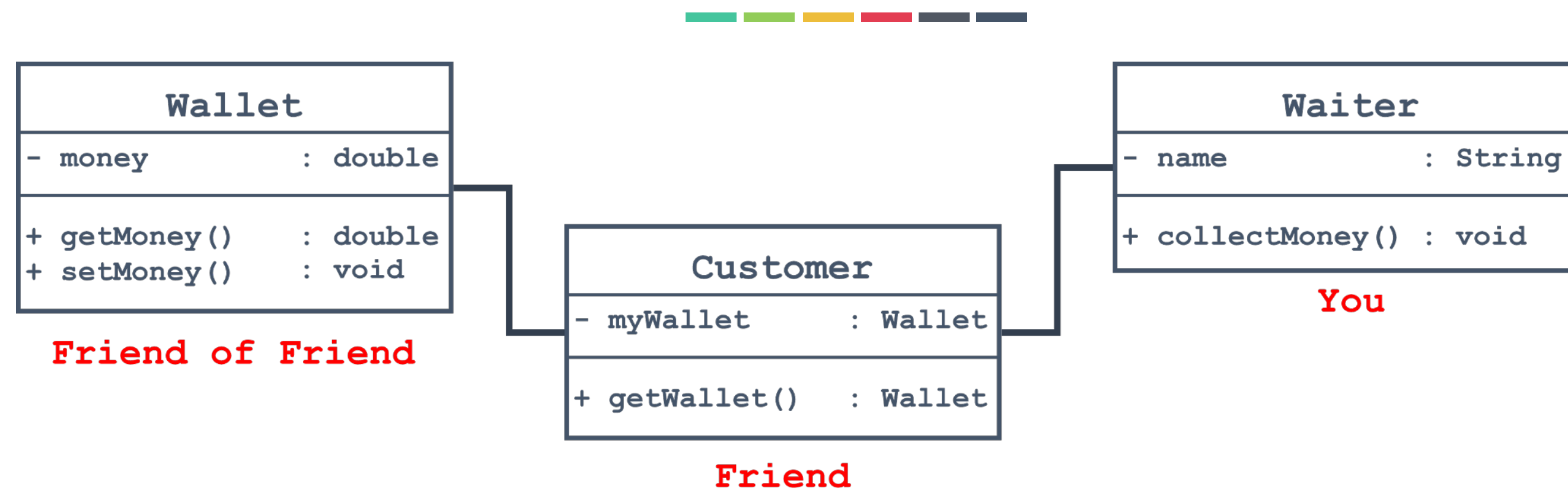
Design Pattern: “*the Law of Demeter*”



the Law of Demeter คือ Design Pattern รูปแบบหนึ่งซึ่งช่วยให้การออกแบบเชิงวัตถุไม่ผูกติดซึ่งกันและกันมากเกินไป อีกทั้งยังสอดคล้องกับ Information Hiding ที่จะซ่อนข้อมูลจากวัตถุอื่น ๆ ที่ไม่ควรทราบหรือเกี่ยวข้องกัน (แค่เพื่อนเท่านั้น)



Design Pattern: “*the Law of Demeter*”



“แต่ละ Object ควรเข้าถึงเพียง Object ที่มันรู้จักเท่านั้น (เพื่อนของมันเท่านั้น) ซึ่งเพื่อนของเพื่อนต้องไม่สามารถเข้าถึงได้” ได้แก่

1. การอ้างอิง**ตัวเอง** เช่น การเรียกใช้ this เป็นต้น
2. การอ้างอิง**ภายในตัวมันเอง** เช่น การเรียกแอตทริบิวต์ การเรียกใช้เมธอด เป็นต้น
3. การ**เรียกใช้ object ที่สัมพันธ์กัน** (โดยตรง) เช่น การใช้พารามิเตอร์ที่ส่งเข้ามาในเมธอด เป็นต้น

```
public class Wallet {
    private double money;
    public double getMoney(){ return this.money; }
    public void setMoney( int m ){ this.money = m; }
}

public class Customer{
    private Wallet myWallet;
    public Wallet getWallet(){ return this.myWallet; }
    public void setWallet(Wallet w){ this.myWallet = w; }
}

public class Waiter{
    private String name;
    private double money;
    public void collectMoney(Customer c, double money){
        if (c.getWallet().getMoney() >= money){
            c.getWallet().setMoney(c.getWallet().getMoney() - money);
            this.money += money;
        }else{
            new Exception("Not Enough!!");
        }
    }
}

public class Restaurant{
    public static void main(String args[]) {
        Wallet w = new Wallet();
        w.setMoney(1000);
        Customer c = new Customer();
        c.setWallet(w);
        Waiter wa = new Waiter();
        wa.collectMoney(c, 840);
    }
}
```



```
public class Wallet {
    private double money;
    public double getMoney(){ return this.money; }
    public void setMoney( int m ){ this.money = m; }
}

public class Customer{
    private Wallet myWallet;
    public Wallet getWallet(){ return this.myWallet; }
    public void setWallet(Wallet w){ this.myWallet = w; }
}

public class Waiter{
    private String name;
    private double money;
    public void collectMoney(Customer c, double money){
        if (c.getWallet().getMoney() >= money){
            c.getWallet().setMoney(c.getWallet().getMoney()-money);
            this.money += money;
        }else{
            new Exception("Not Enough!!");
        }
    }
}
```

ภายในตัวมันเอง และ วัตถุที่สัมพันธ์กัน ✓

ภายในตัวมันเอง และ วัตถุที่สัมพันธ์กัน ✓

วัตถุที่สัมพันธ์กัน ✓

เพื่อนของเพื่อน ✗

ไม่ควรเนื่องจาก

- (1) ปัญหาการผูกติดกันของวัตถุมากเกินไป >> ทำให้โค้ดซับซ้อนเกินไป
- (2) ผิดหลักการ Information hiding เนื่องจาก Waiter รู้ข้อมูล Wallet ของ Customer >> รู้มากเกินไป

```
public class Wallet {
    private double money;
    public double getMoney(){ return this.money; }
    public void setMoney( int m ){ this.money = m; }
}

public class Customer{
    private Wallet myWallet;
    public Wallet getWallet(){ return this.myWallet; }
    public void setWallet(Wallet w){ this.myWallet = w; }
}

public class Waiter{
    private String name;
    private double money;
    public void collectMoney(Customer c, double money){
        if (c.getWallet().getMoney() >= money){
            c.getWallet().setMoney(c.getWallet().getMoney() - money);
            this.money += money;
        }else{
            new Exception("Not Enough!!");
        }
    }
}
```

ภายในตัวมันเอง และ วัตถุที่สัมพันธ์กัน ✓

ภายในตัวมันเอง และ วัตถุที่สัมพันธ์กัน ✓

วัตถุที่สัมพันธ์กัน ✓

เพื่อนของเพื่อน ✗

ไม่ควรเนื่องจาก

- (1) ปัญหาการผูกติดกันของวัตถุมากเกินไป >> ทำให้โค้ดซับซ้อนเกินไป
- (2) ผิดหลักการ Information hiding เนื่องจาก Waiter รู้ข้อมูล Wallet ของ Customer >> รู้มากเกินไป

แก้ไขโดย

ไม่ทำการเรียกใช้หรือเข้าถึงวัตถุที่ไม่ติดกัน หรือไม่เรียกใช้เพื่อนของเพื่อน

```
public class Wallet {
    private double money;
    public double getMoney(){ return this.money; }
    public void setMoney( int m ){ this.money = m; }
    public double withdraw(double m) {
        if (money >= m) {
            money -= m;
        } else { new Exception("Not Enough!!"); }
        return m;
    }
}

public class Customer{
    private Wallet myWallet;
    public Wallet getWallet(){ return this.myWallet; }
    public void setWallet(Wallet w){ this.myWallet = w; }
    public double pay (int m) { return myWallet.withdraw(m); }
}

public class Waiter{
    private String name;
    private double money;
    public void collectMoney(Customer c, double money){
        money += c.pay(money);
    }
}
```